Project - High Level Design

On

Project Title- Multi-Agent Insurance system

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Sarthak Teredesai	EN22CS301879
2.	Shrasti Gupta	EN22CS301925
3.	Shruti Sharma	EN22CS301940
4.	Sanidhya Singh Chouhan	EN22CS301864
5.	Saumya Kumari	EN22CS301886

Group Name: 05D8

Project Number: AAI-40

Industry Mentor Name: Aashruti Shah

University Mentor Name : Prof Pradeep Baniya

Academic Year: 2025-26

Table of Contents

1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. Data Design.
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1. Security Aspects
   - 7.2. Performance Aspects
8. References

1.Introduction

The **Multi-Agent Insurance System** is an intelligent, AI-driven platform designed to automate and optimize various insurance processes using a collaborative agent-based architecture. The system leverages multiple specialized software agents that work together to handle customer queries, policy recommendations, claim processing, risk assessment, and document verification.

Traditional insurance systems often rely heavily on manual intervention, resulting in slower processing times and reduced operational efficiency. This project addresses those challenges by implementing an autonomous multi-agent framework where each agent performs a specific role while communicating with other agents to complete end-to-end insurance workflows.

The system is designed to enhance customer experience, reduce human workload, improve decision-making accuracy, and enable scalable automation in insurance operations.

1.1 Scope of the Document

This document provides a **High-Level Design (HLD)** overview of the Multi-Agent Insurance System. It describes the overall architecture, system components, interactions between agents, and key functional capabilities.

The scope of this document includes:

- Overview of the system architecture
- Description of individual agents and their responsibilities
- High-level workflow of insurance processes (policy recommendation, claims handling, etc.)
- Interaction between agents and external services (LLMs, APIs, databases)
- Key design decisions and technology stack
- Assumptions and system boundaries

1.2 Intended Audience

This document is intended for the following stakeholders:

1. Developers

To understand the architecture, agent roles, and interaction flow before implementing or modifying system components.

2. Project Reviewers / Evaluators

To evaluate the technical design, scalability, modularity, and innovation of the multi-agent approach.

3. System Architects

To review architectural decisions, agent orchestration, and integration patterns.

4. Business Stakeholders

To understand how the system automates insurance workflows and improves operational efficiency.

5. QA / Testing Team

To identify major modules and define test scenarios based on system functionality.

1.3 System Overview

The Multi-Agent Insurance System is built on a **collaborative agent architecture**, where each agent is responsible for a specific task within the insurance lifecycle.

Core Concept

Instead of using a monolithic system, the platform is divided into specialized agents such as:

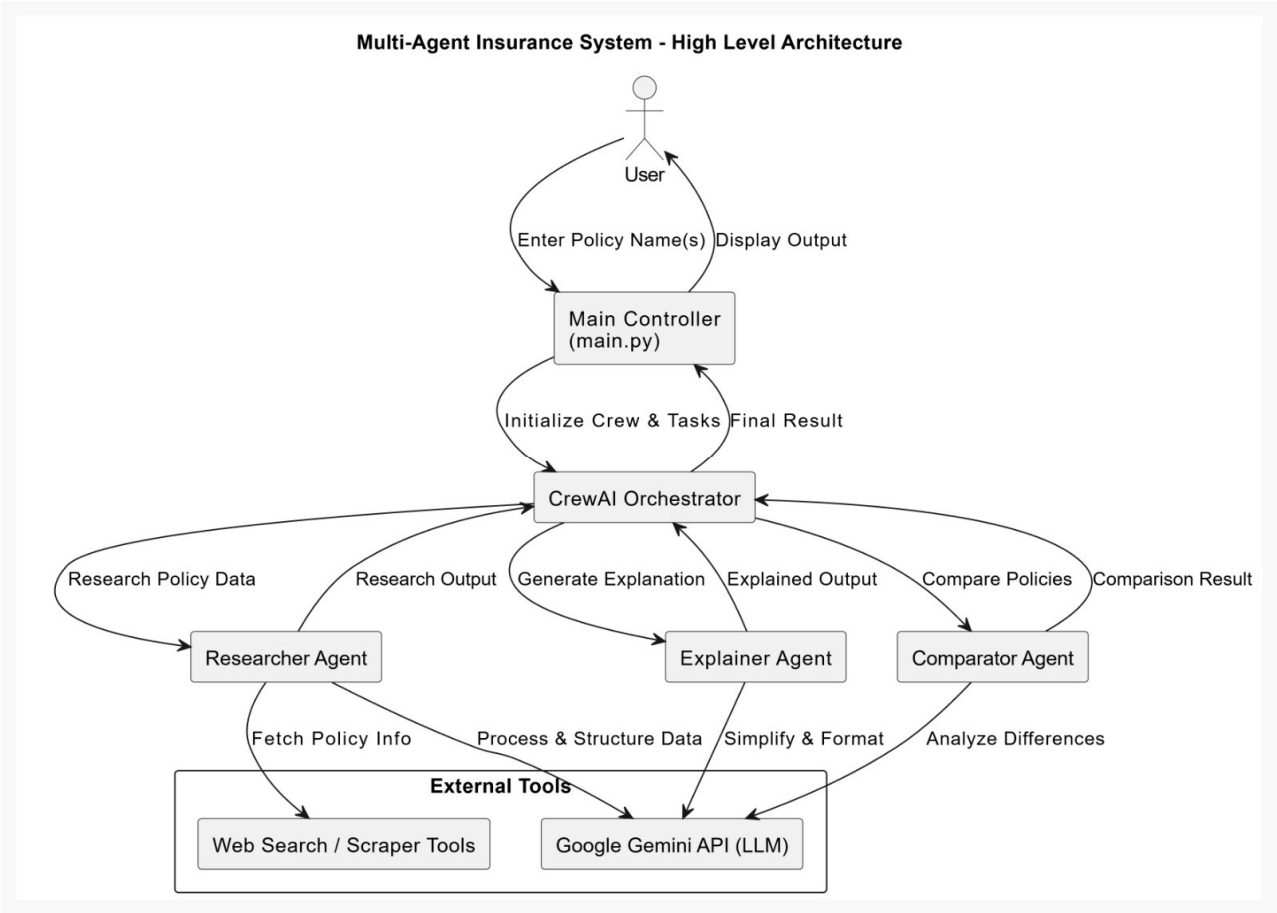
- **Customer Interaction Agent** – Handles user queries and collects customer information.
- **Policy Recommendation Agent** – Suggests suitable insurance plans based on user data.
- **Risk Assessment Agent** – Evaluates customer risk using predefined logic or AI models.
- **Claim Processing Agent** – Manages claim submission, validation, and approval workflows.
- **Document Verification Agent** – Validates uploaded documents.
- **Coordinator Agent** – Manages communication and task delegation among agents.

Each agent operates independently but communicates through structured messages or task delegation mechanisms. This modular structure improves scalability, maintainability, and fault isolation.

High-Level Architecture Flow

1. The user interacts with the system through an interface (CLI/Web API).
2. The Coordinator Agent receives the request.
3. The request is delegated to relevant specialized agents.

4. Agents process tasks independently.
5. Results are aggregated and returned to the user.



Key Features

- Modular multi-agent architecture
- Intelligent policy recommendations
- Automated claim validation
- Risk-based decision logic
- Scalable and extensible design
- Integration with LLMs for natural language interaction

Technology Stack (High-Level)

- Python-based backend
- Multi-agent framework
- LLM integration like Gemini(for intelligent responses)
- REST APIs for communication
- Database for policy and claim data storage

2. System Design

The Multi-Agent Insurance System is designed using a modular and collaborative agent-based architecture. The primary objective of the system design is to create an intelligent, scalable, and extensible platform that automates insurance-related operations such as policy recommendation, risk assessment, claim processing, and document verification. Instead of relying on a monolithic structure, the system is divided into multiple specialized agents that work together under the supervision of a central orchestrator.

The architecture follows a layered approach consisting of a presentation layer, application layer, agent layer, and data layer. This separation ensures maintainability, scalability, and clear responsibility boundaries among components. The design supports easy integration of additional agents in the future without disturbing the existing workflow.

2.1 Application Design

The application is structured around an orchestrator-driven multi-agent model. At the core of the system lies the orchestrator component, which acts as the central controller. It receives user requests, determines the nature of the task, and delegates subtasks to the appropriate agents. Each agent is designed to perform a specialized function independently while communicating results back to the orchestrator.

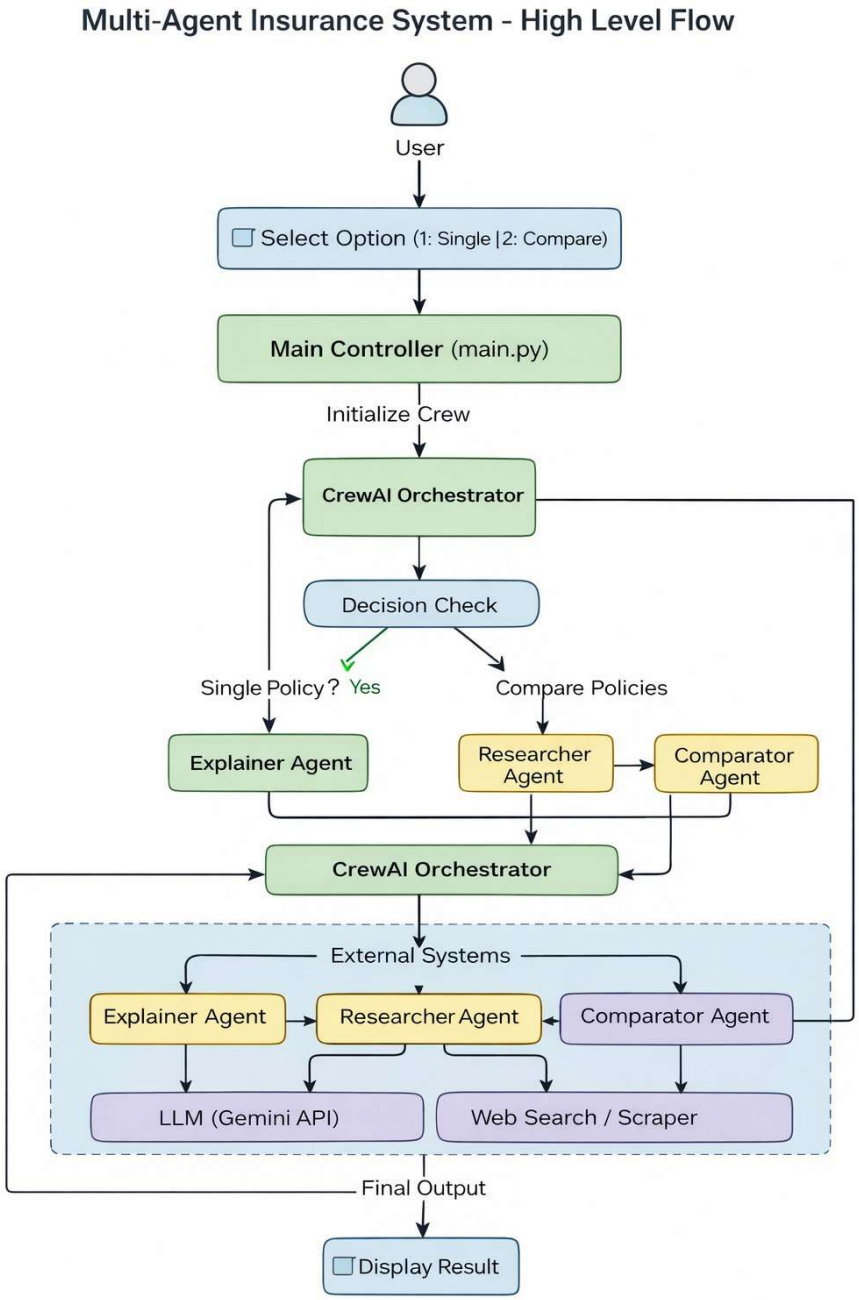
The presentation layer provides an interface through which users interact with the system. This interface may be implemented as a command-line interface, web interface, or REST API endpoint. User inputs such as policy requests or claim submissions are captured and forwarded to the application layer.

Within the application layer, the orchestrator analyzes the request and triggers the necessary agents. For example, a policy recommendation request activates the customer data validation agent, followed by the risk assessment agent and the recommendation agent. The responses from these agents are consolidated and returned to the user in a structured format.

The data layer stores policy information, customer records, and claim histories. This persistent storage ensures data consistency and enables tracking of previous transactions.

2.2 Process Flow

The process flow of the system depends on the type of service requested by the user. In general, every process begins with user interaction and ends with a consolidated decision or recommendation.



In the case of policy recommendation, the user provides personal and financial details. The system validates the input data and evaluates the risk profile of the customer using predefined logic or AI-driven analysis. Based on the calculated risk score and policy constraints, suitable insurance plans are selected and ranked. The orchestrator compiles the final recommendation and presents it to the user.

For claim processing, the user submits claim details along with supporting documents. The document verification agent first ensures that the required documentation is complete and valid. The risk assessment mechanism then evaluates the claim based on policy conditions and risk parameters. The claim processing agent determines whether the claim satisfies eligibility criteria. The final approval or rejection decision is communicated to the user.

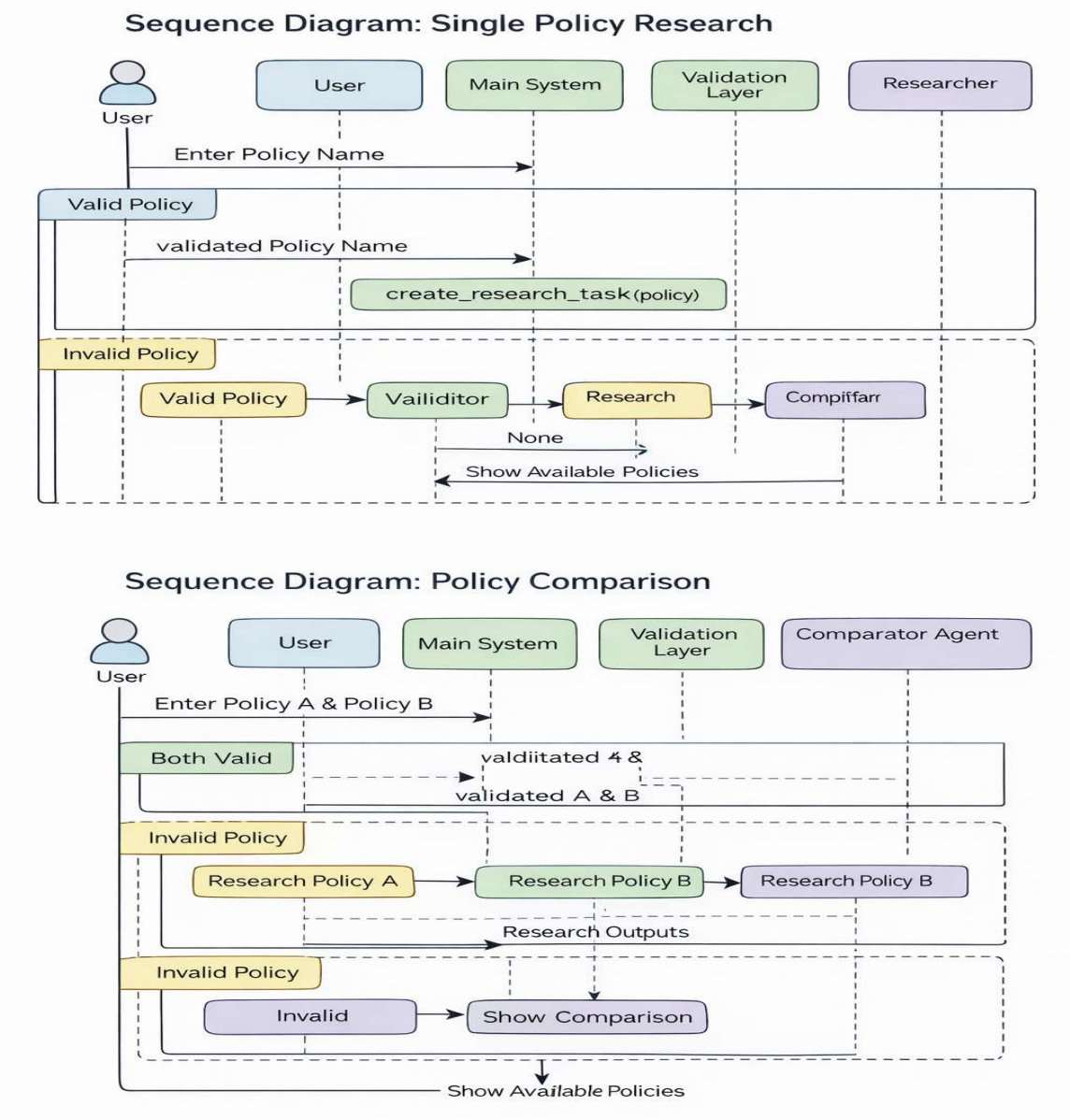
The process flow is designed to ensure logical sequencing of tasks while maintaining independence between agents.

2.3 Information Flow

Information within the system flows in a structured and controlled manner. All user requests are first directed to the orchestrator, which acts as the central communication hub. The orchestrator distributes relevant portions of the information to the required agents. Each agent processes the received input and returns a response to the orchestrator.

The output generated by one agent may serve as input for another. For instance, the risk score generated by the risk assessment agent becomes a key parameter for the policy recommendation or claim processing agent. This sequential information exchange ensures coherent decision-making.

Sensitive data such as customer details and financial information are handled securely. Validation checks are performed before processing to prevent inconsistencies or incomplete submissions. The system ensures minimal redundancy in data handling and maintains logical data dependencies between components.



2.4 Components Design

The system is composed of multiple well-defined components, each responsible for a specific function. The orchestrator component manages workflow execution and ensures proper coordination among agents. It is responsible for initiating tasks, monitoring execution, and aggregating results.

The customer interaction component manages communication with the user and validates input data. The risk assessment component analyzes customer profiles or claims using predefined rules or AI models to determine risk levels. The policy recommendation component matches customer profiles with available insurance plans and prioritizes options according to suitability.

The claim processing component evaluates submitted claims against policy rules and eligibility criteria. The document verification component ensures that uploaded documents are authentic,

complete, and consistent with policy requirements. The database component maintains persistent records of customers, policies, and claims.

Each component is loosely coupled, meaning modifications in one module do not significantly impact others. This design improves maintainability and allows independent development and testing of components.

2.5 Key Design Considerations

The system design prioritizes modularity to ensure that each agent performs a clearly defined role. This separation of concerns enhances readability, maintainability, and future scalability. Scalability is achieved by allowing additional agents or services to be integrated without restructuring the entire architecture.

Extensibility is another important consideration. The design allows new functionalities such as fraud detection or payment integration to be incorporated easily. Fault tolerance is addressed by isolating agent failures so that errors in one component do not compromise the entire system.

Security considerations include validation of user inputs, protection of sensitive data, and controlled access to APIs. The design also ensures that the system remains efficient and responsive under varying workloads.

2.6 API Catalogue

The system exposes a set of REST-based APIs to facilitate communication between external interfaces and the application layer. These APIs allow operations such as customer profile creation, policy recommendation retrieval, claim submission, document verification, and claim status tracking.

For example, a customer profile creation API enables the system to register a new user by capturing essential personal and financial details. The policy recommendation API processes user data and returns a list of suitable insurance plans along with relevant risk information. The claim submission API allows users to initiate claims by providing necessary policy references and supporting documentation.

Each API follows standard HTTP protocols and returns structured JSON responses to ensure consistency and ease of integration. Proper validation and error-handling mechanisms are implemented to ensure reliability and secure communication.

#	Category	API Name	Method	Endpoint	Description	Request Body	Response
1	Authentication	Register User	POST	/api/auth/register	Registers new user (Admin / Customer / Agent)	{ name, email, password, role }	{ message, token }
2	Authentication	Login User	POST	/api/auth/login	Authenticates user and returns JWT	{ email, password }	{ token, role }
3	Policy Management	Get All Policies	GET	/api/policies	Returns list of all insurance policies	—	[{ policy_id, name, provider }]
4	Policy Management	Get Policy Details	GET	/api/policies/:policyId	Returns full structured policy details	—	{ policy_id, name, coverage, premium, benefits[] }
5	Policy Management	Add New Policy (Admin)	POST	/api/policies	Admin adds a new policy	{ name, provider, coverage, premium, benefits[] }	{ message: "Policy added successfully" }
6	Research	Research Single Policy	POST	/api/research	Triggers Researcher Agent to analyze policy	{ policy_name }	{ summary, pros[], cons[] }
7	Research	Explain Policy	POST	/api/explain	Simplifies policy via Explainer Agent	{ policy_id }	{ simple_explanation }

3.Data Design

The Data Design of the Multi-Agent Insurance System defines how information is structured, stored, accessed, maintained, and migrated within the system. Since the application manages sensitive insurance-related data such as customer profiles, policy records, risk scores, and claim details, the data design focuses on integrity, consistency, security, and scalability.

The system follows a structured data management approach where all critical entities are logically organized and stored in a centralized database. The data layer supports the collaborative functioning of agents by ensuring that accurate and consistent information is available whenever required.

The design ensures minimal redundancy, proper relationships between entities, and efficient retrieval mechanisms to support real-time decision-making.

3.1 Data Model

The data model represents the logical structure of the database and defines relationships between different entities in the insurance system. The primary entities in the system include Customer, Policy, Claim, Risk Assessment, and Document.

The Customer entity stores personal and financial details such as name, age, contact information, income level, and medical history. Each customer may be associated with multiple policies. Therefore, a one-to-many relationship exists between Customer and Policy.

The Policy entity contains details about policy type, coverage amount, premium, eligibility conditions, and policy status. A policy is linked to a specific customer and may be associated with multiple claims over time, forming another one-to-many relationship between Policy and Claim.

The Claim entity records information related to claim submissions, including claim amount, reason for claim, submission date, approval status, and associated policy reference. Each claim may have one or more supporting documents stored in the Document entity.

The Risk Assessment entity maintains calculated risk scores and evaluation parameters used during policy recommendation and claim processing. This entity is logically linked to both Customer and Claim records, as risk evaluation may occur during both onboarding and claim review processes.

The data model is designed using relational principles to ensure referential integrity and structured querying. Primary keys uniquely identify each record, while foreign keys establish relationships between entities.

3.2 Data Access Mechanism

The system uses a controlled data access mechanism to ensure secure and efficient interaction with stored information. All data access requests originate from the orchestrator or relevant agents and are routed through a structured data access layer.

The data access layer abstracts direct database operations and provides standardized methods for creating, reading, updating, and deleting records. This abstraction ensures that business logic remains separate from database implementation details.

Agents do not directly manipulate raw database tables. Instead, they invoke service functions that validate input, enforce constraints, and execute secure queries. This approach prevents unauthorized access and reduces the risk of data inconsistencies.

Authentication and validation checks are applied before performing sensitive operations such as claim updates or policy modifications. Transaction management techniques are implemented to ensure that partial failures do not leave the system in an inconsistent state. If an operation fails, the system rolls back changes to maintain data integrity.

Efficient indexing strategies are applied to frequently queried fields such as customer ID, policy ID, and claim ID to improve performance and response time.

3.3 Data Retention Policies

Data retention policies define how long information is stored and how it is managed over time. Since the system deals with insurance and financial data, retention must comply with regulatory and operational requirements.

Customer and policy records are retained for the duration of the policy lifecycle and for a predefined period after policy termination to support audits and dispute resolution. Claim records are preserved to maintain historical transaction data and assist in risk modeling and fraud detection.

Inactive or outdated data may be archived rather than permanently deleted. Archiving ensures that historical data remains accessible for compliance purposes while optimizing database performance.

Sensitive personal information is stored securely and protected through encryption or controlled access mechanisms. When data reaches the end of its retention period, it is securely deleted following defined data disposal standards to prevent unauthorized recovery.

The retention strategy balances regulatory compliance, operational needs, and system performance.

3.4 Data Migration

Data migration refers to the process of transferring data from one system or storage structure to another. The Multi-Agent Insurance System is designed to support migration in scenarios such as system upgrades, database restructuring, or integration with external insurance platforms.

Migration processes are carefully planned to ensure data accuracy and continuity. Before migration, data validation checks are performed to identify inconsistencies or incomplete records. During migration, transformation rules are applied if the new schema differs from the previous structure.

Backup mechanisms are implemented before initiating migration to prevent data loss. After migration, verification procedures ensure that all records have been transferred correctly and relationships between entities remain intact.

The system supports incremental migration strategies where data can be moved in stages rather than all at once. This approach minimizes downtime and reduces operational risks.

Future scalability considerations allow the system to migrate from local storage to cloud-based databases if required, ensuring long-term adaptability and performance optimization.

4.Interfaces

4.1 User Interface

The User Interface enables customers or administrators to interact with the system by submitting policy requests, entering personal details, uploading claim documents, and checking claim status. It may be implemented as a web interface, command-line interface, or API-based client. The interface validates user inputs before sending structured requests to the orchestrator for processing.

4.2 System Interface (API)

The system provides RESTful APIs to connect the user interface with the application layer. These APIs handle operations such as customer registration, policy recommendation, claim submission, and status tracking. All requests and responses follow standardized JSON formats. Authentication and validation mechanisms ensure secure and reliable communication.

4.3 Internal Agent Communication Interface

Agents communicate internally through the orchestrator using structured task messages. The orchestrator distributes requests to relevant agents and collects their outputs. This design maintains loose coupling between agents and ensures smooth coordination in executing workflows.

4.4 Database Interface

The database interface allows agents to access stored data through a controlled data access layer. Direct database manipulation is avoided. All operations such as record creation, updates, and retrieval are performed through defined service methods to maintain data integrity and consistency.

4.5 External System Interface

The system supports integration with external services such as risk evaluation tools or payment gateways if required. Communication with external systems is handled through secure APIs, ensuring validated data exchange and minimal impact on core functionality.

4.6 Interface Design Considerations

The interfaces are designed with a focus on security, reliability, and scalability. Input validation, structured responses, and proper error handling are implemented across all layers. The modular interface structure allows future enhancements without major architectural changes.

5. State and Session Management

State and Session Management in the Multi-Agent Insurance System is designed specifically to support collaborative interaction between three specialized agents: the Researcher Agent, the Comparator Agent, and the Explainer Agent. Since the system focuses on inter-agent communication and structured hand-off protocols, state management plays a critical role in maintaining workflow continuity and ensuring high-quality outputs.

5.1 Session Management

User sessions are handled using secure token-based authentication to maintain continuity during multi-step insurance analysis requests.

- A secure authentication token is generated when a user initiates interaction.
- The token is included in every request for validation.
- Each request is independently authenticated.
- Sessions expire automatically after a defined inactivity period.

This stateless session model ensures that user identity and access control are preserved without storing session data in application memory, enabling scalability and reliability.

5.2 Agent Workflow State Management

The system follows a structured multi-agent workflow where state progresses through defined stages based on agent specialization.

- The **Researcher Agent** gathers and structures insurance policy data such as coverage details, benefits, exclusions, and premium information.
- The **Comparator Agent** evaluates multiple policies using predefined comparison criteria such as cost, coverage value, and suitability.
- The **Explainer Agent** synthesizes the final structured output in a clear and user-friendly format.

Each agent performs its task only after verifying that the required prior state has been completed. The output of one agent becomes the validated input for the next agent, ensuring controlled hand-off and workflow continuity.

5.3 State Lifecycle in Multi-Agent Collaboration

A typical request follows a defined lifecycle:

1. REQUEST_INITIATED
2. DATA_COLLECTED (by Researcher Agent)
3. POLICIES_COMPARED (by Comparator Agent)
4. RESPONSE_GENERATED (by Explainer Agent)
5. COMPLETED

Each stage transition occurs only after successful task completion. This ensures structured progression and prevents incomplete or inconsistent outputs.

5.4 Inter-Agent State Validation and Hand-Off

Since the system demonstrates advanced inter-agent communication, strict state validation rules are enforced:

- An agent cannot execute unless prerequisite states are satisfied.
- Outputs are validated before being passed to the next agent.
- Invalid or out-of-sequence transitions are blocked.
- Errors in one stage do not corrupt previously completed stages.

This controlled hand-off protocol ensures that collaboration between the Researcher, Comparator, and Explainer agents remains consistent and reliable.

5.5 Consistency, Recovery, and Security

To maintain reliability and system integrity:

- Workflow state always remains in the last stable stage if an error occurs.
- Retry mechanisms can be triggered for failed agent execution.
- Role-based access control protects state transitions.
- All state changes are logged for traceability.

These mechanisms ensure that the multi-agent workflow remains structured, secure, and resilient while producing high-quality, well-formatted insurance policy comparisons.

6. Caching

Caching is implemented to improve performance, reduce redundant processing, and optimize response time in the Multi-Agent Insurance System. Since the Researcher Agent gathers policy information and the Comparator Agent evaluates multiple policies, repeated queries may involve similar datasets. Caching helps avoid unnecessary reprocessing of identical or frequently requested information.

6.1 Query-Level Caching

Frequently requested policy information and comparison results are temporarily cached. If a user submits a similar request within a defined time window, the system retrieves the cached response instead of re-triggering the full multi-agent workflow. This reduces latency and improves efficiency.

6.2 Agent Output Caching

Intermediate outputs generated by agents can be cached for reuse during the same session. For example:

- Policy data collected by the Researcher Agent can be reused if comparison parameters change.
- Comparison results generated by the Comparator Agent can be reused if only explanation formatting is modified.

This minimizes repeated computation and LLM calls.

6.3 Cache Expiration and Invalidation

Caching follows controlled expiration policies to ensure data accuracy:

- Cached entries expire after a predefined duration.
- Cache is invalidated if updated policy data is detected.
- Session-specific caches are cleared when the session ends.

These mechanisms ensure that outdated insurance information is not served to users.

6.4 Performance Benefits

The caching strategy provides:

- Reduced response time
- Lower computational overhead
- Optimized agent execution
- Improved scalability under high user load

By reducing redundant multi-agent execution cycles, caching enhances overall system efficiency while maintaining consistency and correctness of insurance policy comparisons.

7. Non-Functional Requirements

Non-Functional Requirements define the quality attributes of the Multi-Agent Insurance System beyond its core functionality. These requirements ensure that the collaborative Researcher, Comparator, and Explainer agents operate securely, efficiently, and reliably under varying conditions.

7.1 Security Aspects

Security is critical as the system processes sensitive insurance-related information and interacts with external data sources.

- Token-based authentication ensures that only authorized users can access the system.
- Role-based access control restricts permissions for different user types.
- Input validation and sanitization prevent injection attacks and malicious inputs.
- Secure communication protocols protect data during transmission.
- Controlled inter-agent communication prevents unauthorized task execution.
- Logging and audit trails track agent actions and state transitions for traceability.

These measures ensure confidentiality, integrity, and controlled access throughout the multi-agent workflow.

7.2 Performance Aspects

Performance requirements focus on ensuring fast response times and efficient multi-agent collaboration.

- Stateless request handling enables horizontal scalability.
- Caching reduces repeated data retrieval and redundant agent execution.
- Optimized hand-off between Researcher, Comparator, and Explainer agents minimizes latency.
- Concurrent request handling supports multiple users simultaneously.
- Controlled retry mechanisms prevent cascading failures.

The system is designed to maintain consistent response quality even under increased workload, ensuring reliable and efficient insurance policy comparison and explanation generation.

8. References

1. Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons.
2. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
3. Jennings, N. R., & Wooldridge, M. (1998). Applications of Intelligent Agents. *Agent Technology: Foundations, Applications, and Markets*. Springer.
4. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine).
5. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
6. OpenAI. (2023). *Large Language Models and Prompt Engineering Best Practices*. Technical Documentation.